

Infografía 1  
OWASP Top 10

Oscar Chavarría Hernández

Universidad CENFOTEC

Gestión de Riesgos

Profesor: JORGE ROBERTO MORA SEGURA

2025

# Inyección (Injection)

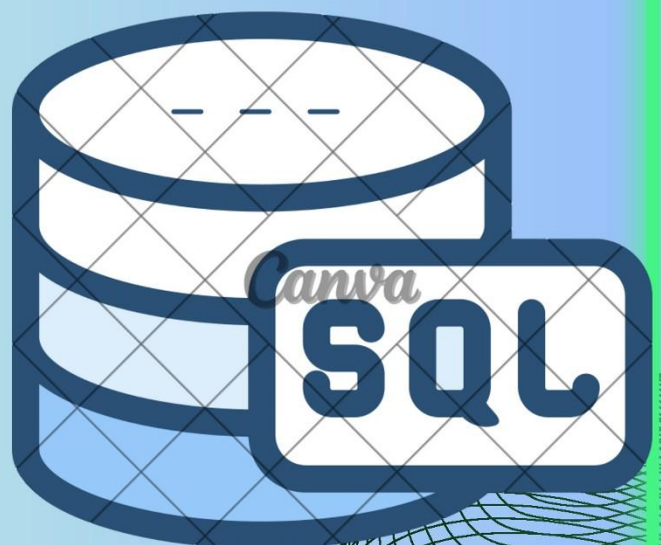
Las inyecciones de código ocurren cuando una aplicación acepta y ejecuta datos maliciosos enviados por un atacante. Estos ataques explotan la falta de validación y sanitización de entradas de usuario para manipular consultas o comandos enviados al backend (bases de datos, sistemas operativos, etc.).

## Ejemplo

- SQL Injection (SQLi): ' OR '1'='1' -- permite obtener acceso no autorizado o extraer datos.
- NoSQL Injection: Inserta comandos en bases como MongoDB para manipular consultas.
- Command Injection: ; rm -rf / ejecuta comandos del sistema.

## Prevención

- Validar y sanitizar entradas.
- Usar consultas preparadas (prepared statements).
- Principio de menor privilegio.





# Autenticación Rota (Broken Authentication)

Fallos en la autenticación y gestión de sesiones permiten a atacantes suplantar a otros usuarios.

## Ejemplo

- Contraseñas simples o por defecto.
- Tokens robados o falsificados.
- Reutilización de sesiones o no cierre de sesión adecuado.

## Prevención

- Implementar autenticación multifactor (MFA).
- Almacenar contraseñas usando hash seguro (bcrypt, Argon2) con sal.
- Establecer tiempos de expiración y revocación de sesiones.





# Exposición de Datos Sensibles (Sensitive Data Exposure)

Los datos sensibles pueden ser expuestos si no se protegen adecuadamente, ya sea durante la transmisión o el almacenamiento. Esto incluye información como números de tarjeta, documentos confidenciales, claves API, etc. Las causas incluyen uso de protocolos inseguros, falta de cifrado, políticas laxas, o errores de configuración.

## Ejemplo

- Transmisión en texto plano (HTTP).
- Almacenamiento sin cifrado (bases de datos, archivos).
- Claves API incrustadas en aplicaciones o repositorios.
- Controles de acceso débiles o mal configurados.

## Prevención

- Cifrar datos en tránsito (TLS/SSL).
- Cifrar datos en reposo con algoritmos robustos.
- Aplicar políticas de privacidad y protección de datos.
- Controlar estrictamente el acceso a datos confidenciales.
- Nunca incluir claves en código visible o repositorios.





# Entidades Externas XML

Los ataques XXE aprovechan el procesamiento inseguro de documentos XML para enviar entidades externas maliciosas. Esto puede permitir a un atacante acceder a archivos internos, realizar ataques DoS o escanear redes internas.

## Ejemplo

- Lectura de archivos locales: Referencias como `file:///etc/passwd`.
- Billion Laughs Attack: Entidades anidadas que consumen recursos y causan denegación de servicio.

## Prevención

- Deshabilitar DTDs y procesamiento de entidades externas.
- Usar bibliotecas de análisis XML seguras y configuradas.
- Validar y sanear entradas XML.

Canva  
XML



# Control de Acceso Roto (Broken Access Control)

Ocurre cuando las restricciones de acceso a recursos o funciones no están bien implementadas, permitiendo a atacantes eludirlas y acceder a datos o acciones no autorizadas

## Ejemplo

- Cambio de roles por manipulación de parámetros (por ejemplo, de usuario a administrador).
- Acceso directo a URL sin control de permisos.
- Modificación de identificadores en formularios o APIs.

## Prevención

- Validar el control de acceso en el servidor.
- Aplicar el principio de menor privilegio.
- Revisar permisos y roles regularmente





# Configuración de Seguridad Incorrecta (Security Misconfiguration)

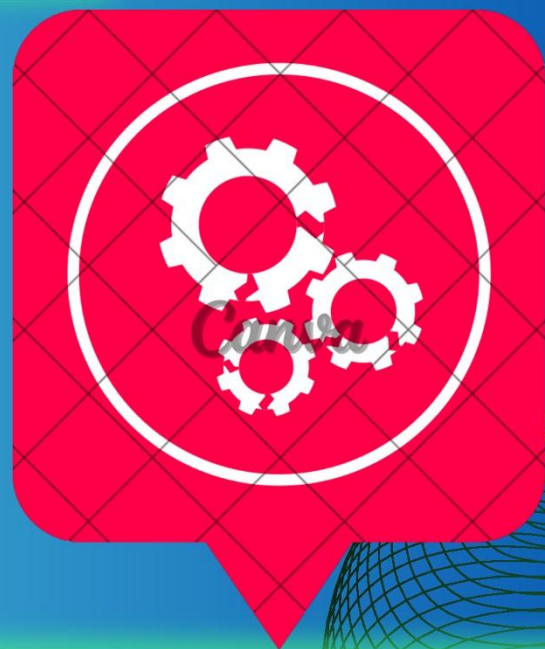
Se produce cuando las configuraciones del sistema, servidor, frameworks o aplicaciones no están aseguradas correctamente, dejando vulnerabilidades que pueden ser explotadas por atacantes.

## Ejemplo

- Directorios y recursos accesibles públicamente.
- Uso de credenciales por defecto.
- Versiones desactualizadas de software.
- Permisos mal configurados.

## Prevención

- Revisar y fortalecer configuraciones.
- Desactivar funciones innecesarias.
- Mantener sistemas y software actualizados.
- Realizar escaneos de seguridad periódicos.





# Cross-Site Scripting (XSS)

Los ataques XSS ocurren cuando una aplicación permite que un atacante inyecte código malicioso (generalmente JavaScript) en las páginas web de otros usuarios, lo que puede llevar a robo de cookies, redirecciones fraudulentas, o control del navegador.

## Ejemplo

- Inyección de formularios falsos para phishing.
- Redirecciones a sitios maliciosos.

## Prevención

- Validar y sanitizar entradas de usuario.
- Escapar las salidas (HTML, JavaScript, CSS).
- Implementar políticas CSP (Content Security Policy).





# Deserialización Insegura (Insecure Deserialization)

Ocurre cuando se deserializan datos no confiables sin validación, lo que permite a atacantes inyectar contenido malicioso o modificar la lógica del programa para ejecutar código arbitrario.

## Ejemplo

- Ejecución remota de código (RCE) mediante objetos manipulados.
- Elevación de privilegios mediante modificación de datos serializados.
- Inyección de lógica maliciosa en flujos de la aplicación.

## Prevención

- Validar y restringir los datos que se deserializan.
- Usar formatos seguros o firmar los datos serializados.
- Evitar la deserialización de fuentes no confiables.





## Uso de Componentes con Vulnerabilidades Conocidas (Using Components with Known Vulnerabilities)

Ocurre cuando se utilizan bibliotecas, frameworks o componentes de terceros que tienen vulnerabilidades conocidas y no se han actualizado, lo que abre puertas a posibles ataques.

### Ejemplo

- Uso de versiones antiguas de jQuery, Apache Struts, etc.
- Plugins o dependencias con CVEs conocidos.
- Falta de parches de seguridad.

### Prevención

- Mantener todas las dependencias actualizadas.
- Usar herramientas de análisis de vulnerabilidades.
- Evitar componentes abandonados o sin soporte.





# Registro Y Monitoreo Insuficientes (Insufficient Logging & Monitoring)

La falta de registros o monitoreo adecuado dificulta la detección de actividades maliciosas y la respuesta oportuna a incidentes. Sin monitoreo, los ataques pueden pasar desapercibidos y causar daños prolongados.

## Ejemplo

- No registrar intentos fallidos de inicio de sesión.
- No configurar alertas para actividades sospechosas.
- No contar con análisis forense tras incidentes.

## Prevención

- Registrar eventos críticos (accesos, cambios, errores).
- Configurar alertas automáticas para comportamientos anómalos.
- Analizar registros regularmente para identificar patrones.





# Reflexión final

Reflexione y responda las siguientes preguntas.

- 1. ¿Qué elementos visuales considera más efectivos para representar los riesgos de seguridad en sus infografías?  
¿Por qué?

Me parecen más útiles los iconos sencillos y que representen claramente cada riesgo, como los típicos candados, alertas o símbolos de usuario. Estos ayudan a que cualquiera pueda asociar rápidamente la imagen con el concepto. También es muy importante usar colores que transmitan el nivel de riesgo, por ejemplo, rojo para peligro o amarillo para advertencia. Los diagramas de procesos y resúmenes cortos ayudan mucho a explicar cómo ocurren estos problemas de forma rápida y fácil.

- 2. ¿Cómo asegurar que la infografía sea clara y comprensible para una audiencia que no está familiarizada con las vulnerabilidades de seguridad?

Lo principal es hablar claro y evitar términos complicados. Hay que explicar cada riesgo con palabras simples y dar ejemplos fáciles de entender, como lo que pasa con contraseñas débiles o datos filtrados. Es importante usar un diseño que esté bien organizado, con secciones diferenciadas y encabezados claros para que cualquiera pueda seguirlo. Además, si hay muchos temas, lo mejor es dividirlos en varias partes para que no sea abrumador.

- 3. ¿Qué desafíos encontró al sintetizar la información técnica sobre las vulnerabilidades en un formato visual y cómo los superó?

El reto más grande fue simplificar conceptos técnicos sin que se pierda su significado. Lo que hice fue elegir ejemplos fáciles de identificar que mostraran de forma clara el riesgo, sin entrar en detalles complicados.

- 4. ¿Cómo podría mejorar su infografía para hacerla más atractiva o informativa?

Se podría hacer más llamativa agregando animaciones o enlaces interactivos, para que la gente pueda profundizar en cada punto si le interesa. También sería útil sumar gráficos que muestren estadísticas reales, como qué vulnerabilidades son más comunes.